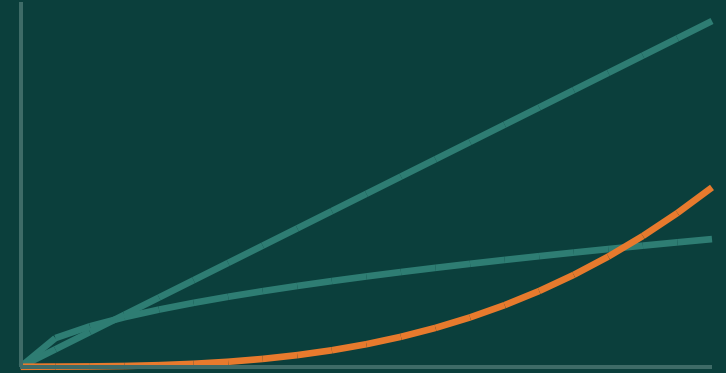




ESTRUTURAS DE DADOS E ALGORITMOS

Complexidade Temporal de Algoritmos

Aula 2 · como prever o tempo antes de correr o código

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

O que é um algoritmo? Diga-o por palavras suas.

2

Qual a diferença entre um algoritmo e um programa?

3

Quantos passos precisa a pesquisa binária numa lista ordenada de 1 000 000 de nomes?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Da intuição à notação Big O

1

Porque importa

O tempo cresce com os dados — às vezes de forma explosiva.

2

Contar operações

Medir sem cronómetro: a função $T(n)$.

3

Notação Big O

Ficar só com o que manda no crescimento.

4

Pesquisa binária e tipos de problemas

$O(\log n)$ na prática; problemas tratáveis e intratáveis.

Pergunta-guia: como prever quanto tempo um algoritmo vai demorar, sem o executar?

O que torna um algoritmo "bom"?

FOCO DE HOJE

Eficiência temporal

Quanto tempo demora?

Eficiência espacial

Quanta memória gasta?

Simplicidade

É fácil de perceber e de verificar?

Generalização

Resolve um problema mais geral?

O cronômetro engana: o tempo medido depende da máquina, da linguagem e dos dados de teste. Queremos uma medida que dependa apenas do algoritmo e do tamanho da entrada, n .

1

PARTE 1

Porque importa o tempo

Quando os dados crescem, o algoritmo decide tudo.

Um computador faz 10^9 operações por segundo...

Quanto tempo demora um algoritmo que executa $f(N)$ operações?

$f(N)$	$N = 100$	$N = 1\ 000$	$N = 10\ 000$
N	< 0,01 s	< 0,01 s	< 0,01 s
N^2	< 0,01 s	< 0,01 s	0,1 s
N^3	< 0,01 s	1 s	16 min
N^4	0,1 s	16 min	115 dias
2^N	10^{13} anos	10^{284} anos	10^{2993} anos
$N!$	10^{141} anos	10^{2551} anos	10^{35642} anos

Para ter noção

O Universo tem cerca de $1,4 \times 10^{10}$ anos.

Um algoritmo 2^N com $N = 100$ demoraria **mais de mil vezes** a idade do Universo.

EXEMPLO

Gerar todas as permutações de N números

Há **N!** permutações de N números. Um computador gera 10^9 permutações por segundo. É exequível?

N = 10

3,6

milissegundos

N = 18

74

dias

N = 19

3,9

anos

N = 20

77

anos

Acrescentar um único número multiplica o tempo por N. Passar de 18 para 20 números transforma 2 meses e meio em 77 anos.

Comprarmos um computador 1000 vezes mais rápido?

	Computador atual		1000× mais rápido
N = 20	77 anos	→	28 dias
N = 21	1 619 anos	→	1,6 anos
N = 22	35 615 anos	→	36 anos

Moral: mil vezes mais hardware só nos deixa acrescentar dois números. Um melhor algoritmo vale mais do que uma máquina melhor.

2

PARTE 2

Contar operações

Medir o tempo sem cronómetro.

Quanto demora um download?

Estabelecer a ligação demora 5 s; depois, cada 70 MB demora 1 s.

$$T(n) = 5 + n / 70$$

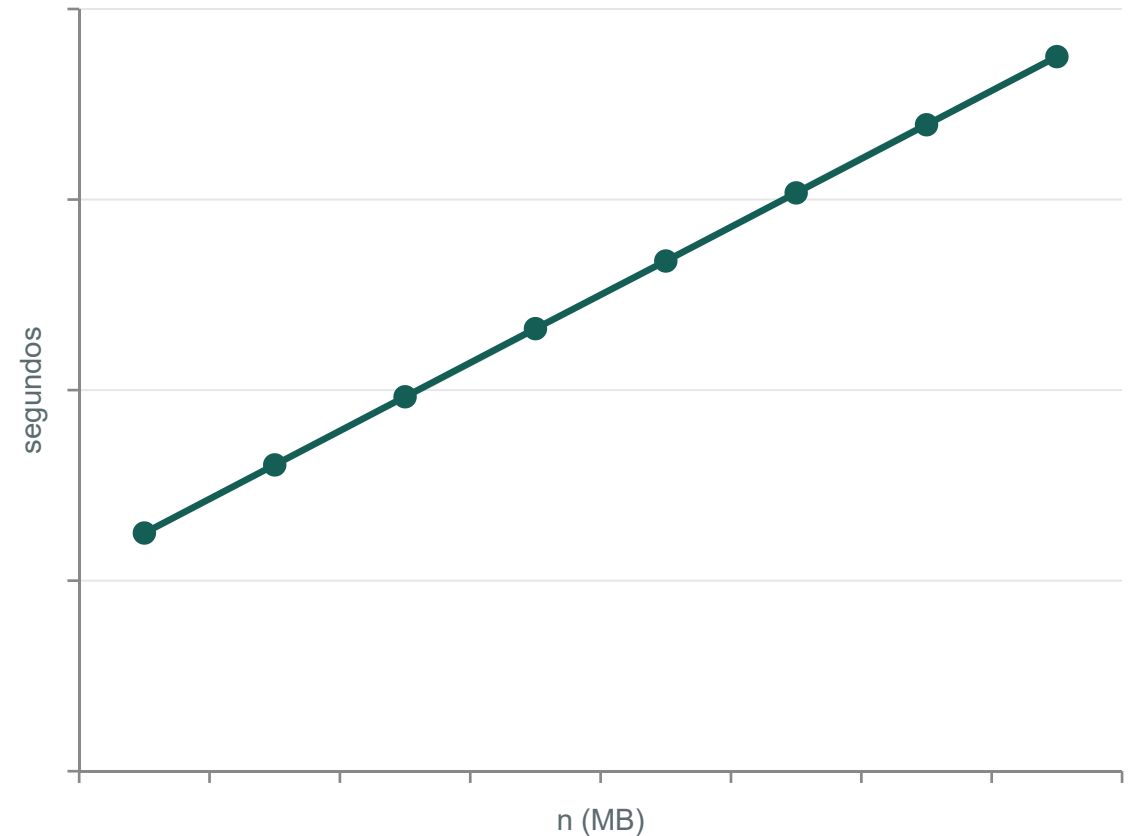
5 = custo fixo (não depende de n)

$n / 70$ = custo que cresce com n

O tempo pode medir-se em segundos, em passos, em operações...

Duplicar o ficheiro ≈ duplicar o tempo → linear

Tempo de download (s) em função do tamanho n (MB)



Contar as operações de um ciclo

```
int count = 0;
for (int i = 0; i < n; i++)
    if (v[i] == 0) count++;
```

Conta os zeros de um array v com n elementos.

Pior caso *(todos os valores são 0)*
 $T(n) = 2 + 2 + (n+1) + n + n + 2n = 5 + 5n$

Melhor caso *(nenhum valor é 0)*
 $T(n) = 2 + 2 + (n+1) + n + n + n = 5 + 4n$

Operação	Tipo	Veze
int count = 0	declaração + atribuição	2
int i = 0	declaração + atribuição	2
i < n	comparação "menor que"	n + 1
v[i]	acesso ao array	n
== 0	comparação "igual a"	n
i++	incremento	n
count++	incremento (só se v[i] == 0)	0 ... n

Pior caso, caso médio e melhor caso

VAMOS USAR SEMPRE ESTE

Pior caso

Tempo máximo para qualquer entrada de tamanho n .

Trânsito na hora de ponta: é com este que planeamos.

ÚTIL, MAS DIFÍCIL

Caso médio

Tempo médio sobre todas as entradas de tamanho n .

Um dia normal. Útil, mas exige saber como são os dados.

ENGANADOR

Melhor caso

A entrada para a qual o algoritmo é mais rápido.

Domingo às 7 da manhã: bonito, mas enganador.

3

PARTE 3

Notação Big O

Ficar só com o que manda no crescimento.

Quando n é grande, só o maior termo importa

1

Descreve a taxa de crescimento
não o número exato de operações.

2

Fica o termo de maior grau
os de ordem inferior desaparecem.

3

Caem as constantes
e fica a expressão mais simples.

$$T(n) = 5 + 5n \longrightarrow O(n)$$

$$T(n) = 5 + 4n + 3n^2 \longrightarrow O(n^2)$$

Quem manda em $T(n) = 5 + 4n + 3n^2$?

n	5	4n	3n ²	peso de 3n ²
10	5	40	300	87 %
100	5	400	30 000	98,7 %
1 000	5	4 000	3 000 000	99,9 %

À medida que n cresce, o termo quadrático "engole" os restantes. É por isso que os podemos ignorar.

Do melhor para o pior

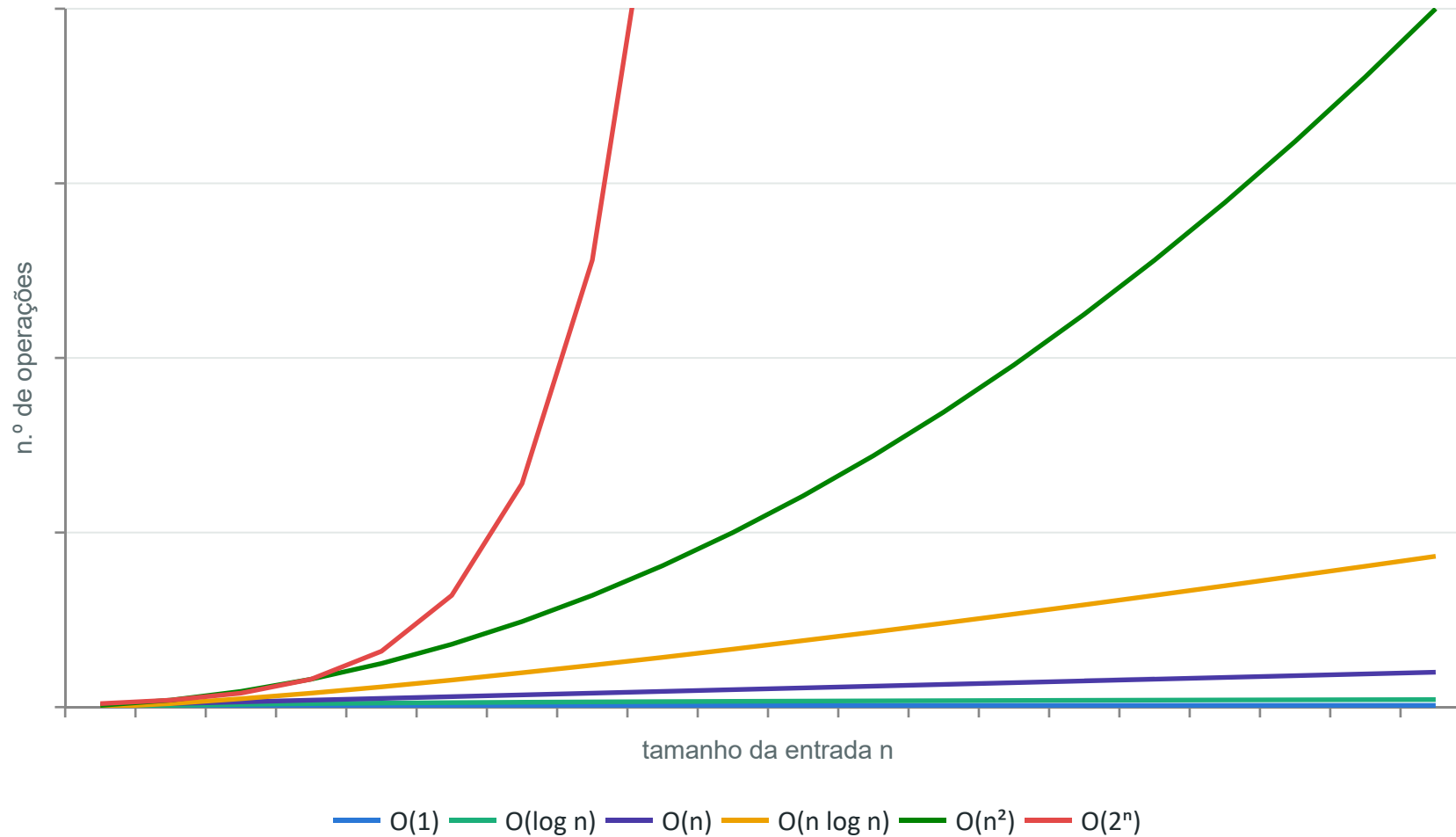
$O(1)$	Constante	Aceder a $v[5]$ num array
$O(\log n)$	Logarítmica	Pesquisa binária
$O(n)$	Linear	Pesquisa sequencial
$O(n \log n)$	Log-linear	Boas ordenações (ex.: mergesort)
$O(n^2)$	Quadrática	Dois ciclos encadeados
$O(n^3)$	Cúbica	Três ciclos encadeados
$O(2^n)$	Exponencial	Testar todos os subconjuntos

mais rápido

A vermelho: impraticável quando N chega às centenas ou milhares.

mais lento

Como cresce cada função



Com $n = 20$

$O(1)$	1
$O(\log n)$	≈ 4
$O(n)$	20
$O(n \log n)$	≈ 86
$O(n^2)$	400
$O(2^n)$	1 048 576

Qual é a complexidade (no pior caso)?

- A Procurar um elemento num array $O(?)$
- B Obter o elemento no índice 5 de um array $O(?)$
- C ... e de um ArrayList? $O(?)$
- D ... e de uma LinkedList? $O(?)$
- E Pesquisa binária num array ordenado $O(?)$

Discuta 2 minutos com o colega do lado antes de ver as respostas.

Qual é a complexidade? As respostas

A	Procurar um elemento num array	$O(n)$	Pode ter de ver todos os elementos.
B	Índice 5 de um array	$O(1)$	Acesso direto: endereço = início + 5 × tamanho.
C	Índice 5 de um ArrayList	$O(1)$	Por dentro é um array.
D	Índice 5 de uma LinkedList	$O(n)$	Tem de seguir as ligações nó a nó.
E	Pesquisa binária	$O(\log n)$	Divide o problema ao meio em cada passo.

Atenção: para algoritmos recursivos é preciso outra ferramenta — as equações de recorrência.

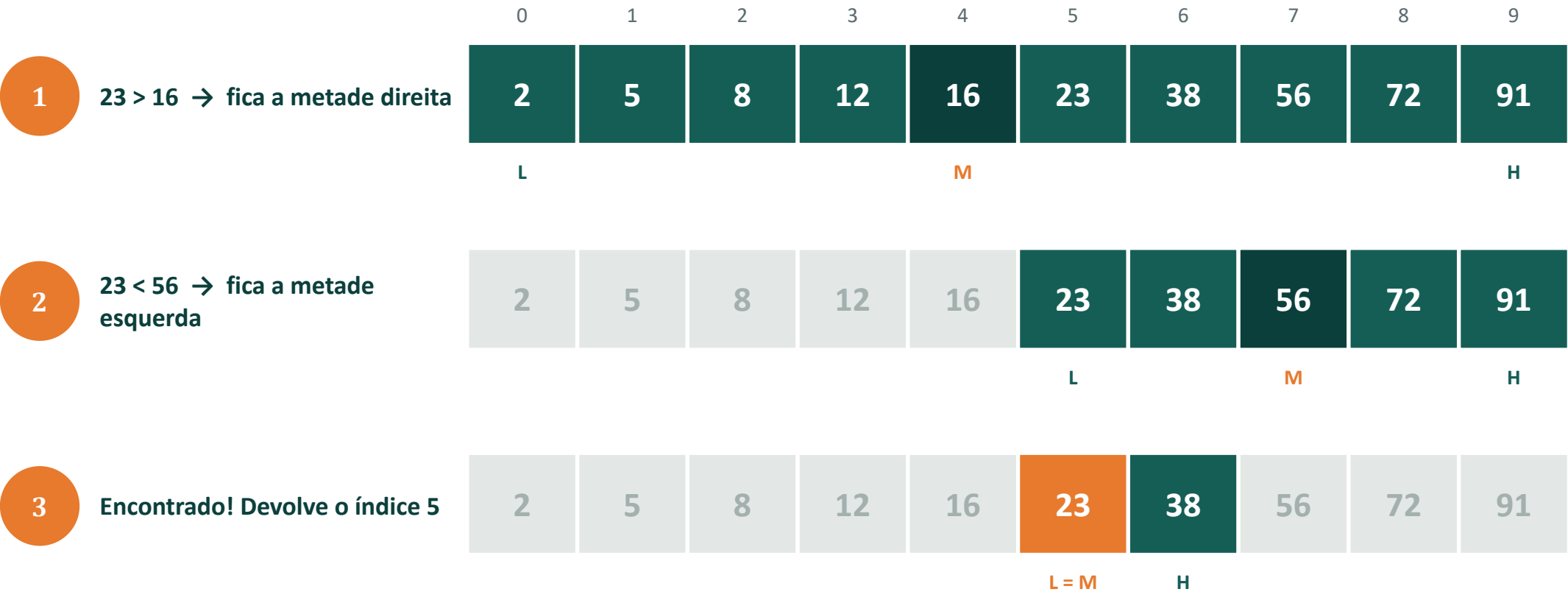
4

PARTE 4

Pesquisa binária e tipos de problemas

$O(\log n)$ em ação — e os limites do que é possível.

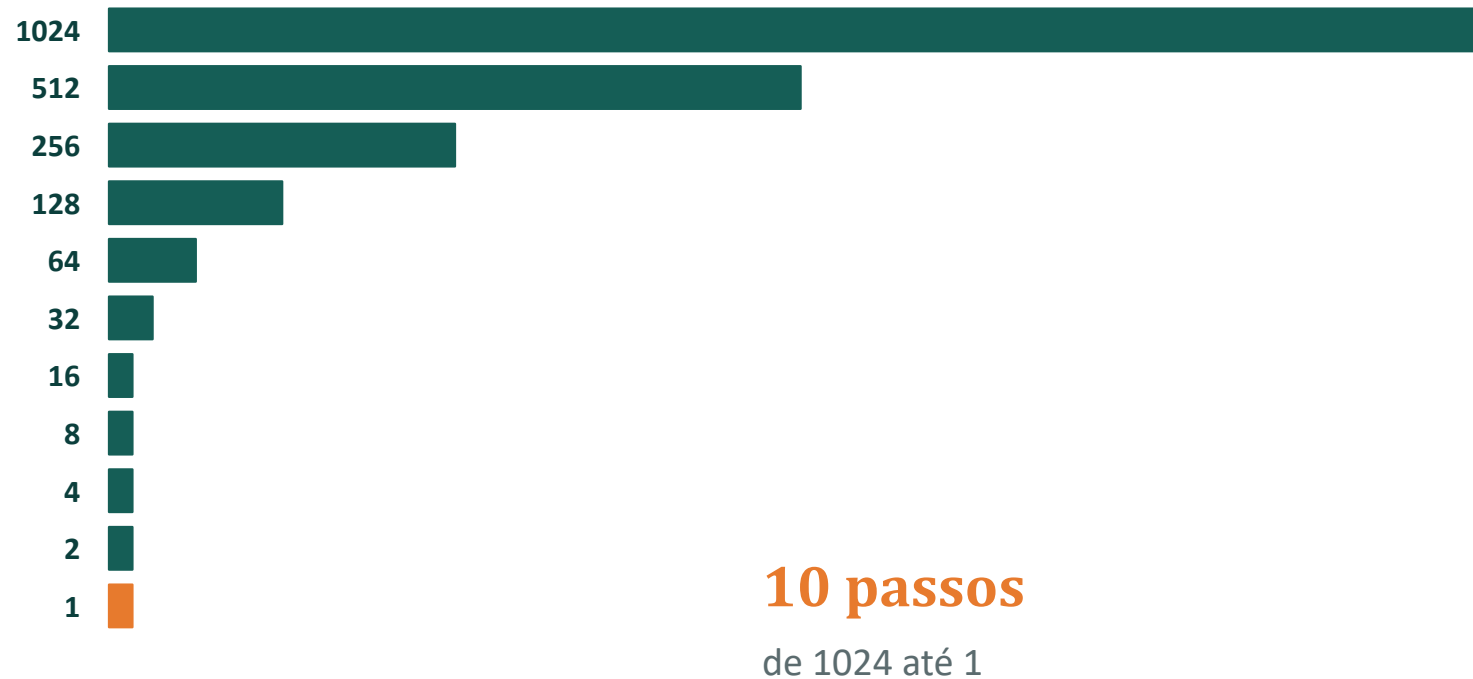
Encontrar o 23 em três passos



Requisito: o array tem de estar ordenado. Cinza = já descartado.

Porque é que a pesquisa binária é $O(\log n)$?

Cada comparação corta ao meio os elementos que ainda podem conter a resposta:



Regra prática

$n = 1\,000$

≈ 10 passos

$n = 1\,000\,000$

≈ 20 passos

$n = 10^9$

≈ 30 passos

Multiplicar n por 1000 só acrescenta cerca de 10 passos.

Tratáveis e intratáveis

Problemas tratáveis

classe P

- Têm soluções que escalam: o tempo não explode.
- No pior caso são $O(N^k)$, com k tipicamente 2 ou 3.
- Ex.: ordenar, pesquisar, caminho mais curto.

Problemas intratáveis

ex.: problemas NP-completos

- Só se conhecem soluções com complexidade exponencial.
- Ex.: caixeiro-viajante, horários, mochila.
- Pouco mais de $N = 50$ já pode ser inviável.

Como enfrentar um problema intratável?

Aproximação

Aleatorização

Restrições

Parametrização

Heurísticas

Agora é consigo

1 Pesquisa binária

Implemente em Java o algoritmo da pesquisa binária, com o protótipo:

```
int binarySearch(int[] arr, int target)
```

Devolve o índice de target, ou **-1** se não existir.

Teste com o array do exemplo: 23 → 5; 40 → -1.

2 Máxima subsequência contígua

Dada uma sequência de inteiros (possivelmente negativos), encontre a subsequência contígua com a maior soma. Se todos forem negativos, a soma é 0.

-2	11	-4	13	-5	2
----	----	----	----	----	---

 = 20

1	-3	4	-2	-1	6
---	----	---	----	----	---

 = 7

Há soluções $O(N^3)$, $O(N^2)$ e $O(N)$. Consegue chegar à linear?

Três ideias a reter

1

O algoritmo manda

Um algoritmo melhor vale mais do que um computador mais rápido. $N!$ e 2^N explodem.

2

Contar operações

$T(n)$ conta operações em função de n .
Analisamos sempre o pior caso.

3

Big O simplifica

Fica o termo dominante, sem constantes: $5 + 4n + 3n^2$ é $O(n^2)$.

Próximo passo: analisar algoritmos recursivos com equações de recorrência.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.ª ed.). Pearson.

2

Weiss, M. A. (2010). *Data Structures and Problem Solving Using Java* (4.ª ed.). Pearson.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt